

Lab sheet 21**Submission Due: 06.11.2024****Title:** E – Commerce App (Laravel) - II**Aims:**

- Implement Laravel backend application

Tasks:

- Create a New Project
- Create model, migrations controller and request
- Implement CRUD operation
- Test with postman

Activities:

1. Create new project named “e-commerce-app”
2. Create model & migration

php artisan make:model Product -m

3. Create controller

php artisan make:controller ProductController

4. Create request

php artisan make:request CreateProductRequest

5. Update .env file
6. Complete Product model

```
class Product extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'description',
        'image',
        'price',
    ];
}
```

7. Complete migrations

```
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description');
        $table->string('image');
        $table->decimal('price', 10, 2);
        $table->timestamps();
    });
}
```

8. Run the migrations

9. Implement CRUD operation

```
public function all()
{
    return Product::all();
}

public function store(CreateProductRequest $request)
{
    $imagePath = $request->file('image')->store('products',
'public');

    $product = Product::create([
        'name' => $request->validated('name'),
        'description' => $request->validated('description'),
        'price' => $request->validated('price'),
        'image' => $imagePath,
    ]);
    return $product;
}

public function get($id)
{
    return Product::findOrFail($id);
}

public function update(Request $request, $id)
{
    $product = Product::findOrFail($id);
    return $product->update($request->all());
}
```

```
public function delete($id)
{
    $product = Product::findOrFail($id);
    return $product->delete();
}
```

10. Complete request
11. Implement routes
12. Implement new route for get token

```
Route::get('/token', function () {
    return csrf_token();
});
```

13. Open postman
14. Run the token route
15. Save token in header
16. Run the requests

The screenshot shows the Postman interface for a new request. The URL is `localhost:8000/product/store` and the method is `POST`. The `Headers` tab is active, showing a table with one header: `X-CSRF-TOKEN` with a value starting with `9T5LKTtMug9QPEoBbRkUE6buFXtPBz6uTB...`. The `Body` tab is also active, showing a JSON response in the `Pretty` view:

```
{
  "name": "Product 1",
  "description": "This is product 01",
  "price": 23,
  "image": "",
  "updated_at": "2024-10-29T17:09:45.000000Z",
  "created_at": "2024-10-29T17:09:45.000000Z",
  "id": 2
}
```

The status bar at the bottom indicates a `201 Created` response with a status of 201, a response time of 801 ms, and a size of 1.27 KB.

The screenshot shows a REST client interface with a dark theme. At the top, it says "New Collection / New Request". The request method is "POST" and the URL is "localhost:8000/product/store". The "Body" tab is selected, showing a raw JSON request:

```
1 {
2   "name": "Product 1",
3   "description": "This is product 01",
4   "image": "",
5   "price": 23
6 }
```

Below the request, the "Body" tab of the response is selected, showing a pretty-printed JSON response:

```
1 {
2   "name": "Product 1",
3   "description": "This is product 01",
4   "price": 23,
5   "image": "",
6   "updated_at": "2024-10-29T17:09:45.000000Z",
7   "created_at": "2024-10-29T17:09:45.000000Z",
8   "id": 2
9 }
```

The response status is "201 Created" with a time of 801 ms and a size of 1.27 KB. There are also buttons for "Save Response" and "Beautify".